

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Algorithm Design Challenge

**COURSE NAME: ARTIFICIAL
INTELLIGENCE AND EXPERT
SYSTEMS**

**COURSE OUTCOME COVERED
COURSE CODE: MLA01**

CO1: *Apply search algorithms and heuristic techniques to solve state-space search and optimization problems. (BTL 3)*

Assessment Tool Weightage: 40 %

Total Marks: 100 Time: 90 Mins No. of Questions: 5

Annexure A

Algorithm Design Challenge

Code of Conduct:

I Kamalli shree V (192525030) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Criterion	Weightage	Marks Obtained
Problem Analysis and Understanding	15%	
AI Algorithm Selection & Justification	15%	
Algorithm Design / Pseudocode	25%	
Application of AI Concepts (Greedy, A*, CSP, Minimax, etc.)	15%	
Diagram / State Space / Search Tree Representation	10%	
Complexity Analysis and Solution Efficiency	10%	
Originality, Critical Thinking, and Presentation	10%	
Total	100%	

Signature of the Faculty Total Marks Awarded
Answer all the Questions

Q.No	Question	Topics
1	Design an algorithm using Greedy Best-First Search to find a path from a source node to a goal node for the given RomaniaMap Problem graph. Clearly define the heuristic function, show the node expansion order, and justify why Greedy Best-First Search is suitable for this problem.	Informed Search, Greedy Best-First Search, Heuristic Functions
2	A delivery robot must travel from a warehouse to a customer through a weighted road network. Design an A Search algorithm to determine the optimal path. Calculate $g(n)$, $h(n)$, and $f(n)$ for each expanded node, and explain why A guarantees the optimal solution.	A* Search, Heuristic Functions
3	A university needs to prepare an examination timetable without scheduling conflicting subjects at the same time. Design the problem as a Constraint Satisfaction Problem (CSP) by identifying variables, domains, and constraints. Explain how MRV and Forward Checking improve the efficiency of the solution.	CSP, MRV, Forward Checking
4	Design the Minimax algorithm for the given game tree to determine the best move for the MAX player. Then apply Alpha-Beta Pruning to the same tree and indicate which nodes are pruned. Compare the number of nodes evaluated before and after pruning.	Game Playing, Minimax, Alpha-Beta Pruning
5	A warehouse robot operates in a dynamic environment where obstacles may appear unexpectedly. Design an intelligent search strategy using either Local Search or an Online Search Agent. Explain how the algorithm adapts to environmental changes and compare its performance with informed search algorithms.	Local Search, Optimization, Online Search Agents

CO2: ASSESSMENT TOOL – 1

NAME: KAMALLI SHREE V

REG.NO: 192525030

**COURSE: MLA 0103 – ARTIFICIAL INTELLIGENCE
AND EXPERT SYSTEMS**

Q1. Greedy Best-First Search – Romania Maze

Problem Statement

Find a path from Arad to Bucharest using Greedy Best-First Search.

Problem Analysis

Initial: Arad , Goal: Bucharest , State: City

Action: Travel through roads

AI Technique

Greedy Best-First Search

$$f(n)=h(n)$$

Justification

GBFS selects the city closest to the goal according to the heuristic.

State Space

Arad → Sibiu → Fagaras → Bucharest

Variables & Constraints

Variable: Current city

Domain: Connected cities

Constraint: Avoid repeated/invalid states.

Heuristic

Straight-line distance to Bucharest.

Node	$h(n)$
Arad	366
Sibiu	253
Fagaras	176
Bucharest	0

Pseudocode

1. Add start to OPEN.
2. Select node with minimum $h(n)$.
3. If goal → return path.
4. Expand node.
5. Add successors.

Execution

Arad → Sibiu → Fagaras → Bucharest

Cost: $140+99+211=450$ km

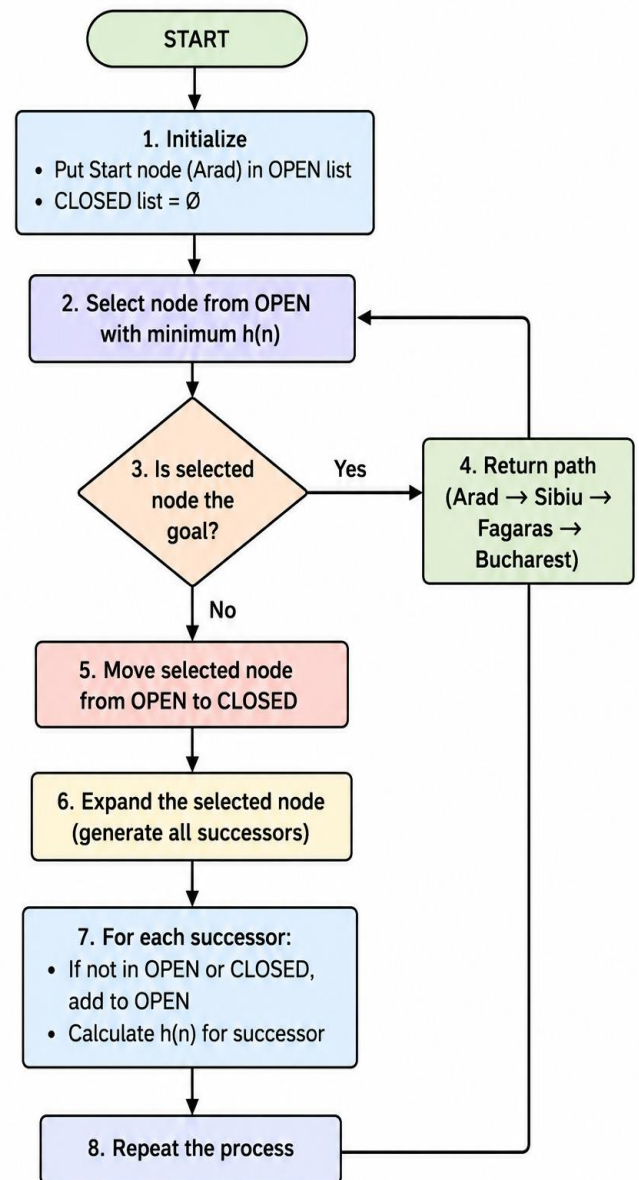
Complexity

Time: $O(b^m)$

Space: $O(b^m)$

Conclusion

GBFS reaches the goal quickly using heuristic guidance, but does not guarantee the shortest path.



Q2. A* Search – Delivery Robot

Problem Statement

Find the minimum-cost route for a delivery robot from Warehouse to Goal using A*.

Problem Analysis

Initial: Warehouse , Goal: Customer

State: Location , Cost: Road weight

AI Technique

A* Search - $f(n)=g(n)+h(n)$

Justification

- A* considers both travelled cost and estimated remaining cost.
- With an admissible heuristic, it gives an optimal solution.

State Space : Warehouse $\rightarrow$ B $\rightarrow$ D $\rightarrow$ Goal

Variables & Constraints

Variable: Robot location

Domain: Connected locations

Constraint: Follow valid roads.

Heuristic

Node	$h(n)$
Warehouse	9
B	4
D	3
Goal	0

Pseudocode

- Set $g(\text{Start})=0$.
- Calculate $f=g+h$.
- Select minimum f .
- If Goal $\rightarrow$ return path.
- Expand and update costs.

Execution

Node	g	h	f
Warehouse	0	9	9
B	4	4	8
D	5	3	8
Goal	8	0	8

Optimal Path:

Warehouse $\rightarrow$ B $\rightarrow$ D $\rightarrow$ Goal , Cost = 8

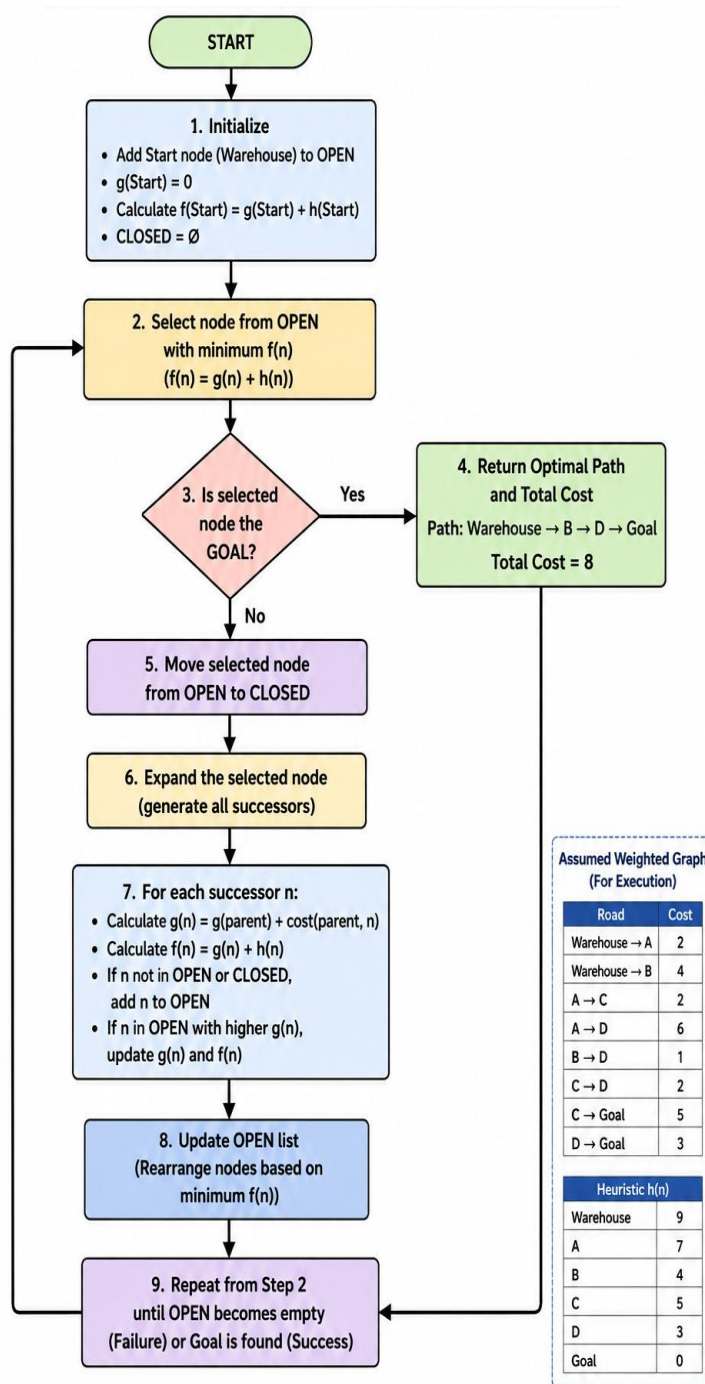
Complexity

Time: $O(b^d)$

Space: $O(b^d)$

Conclusion

A* finds the optimal route using $g(n)+h(n)$.



Q3. Examination Timetable Using CSP

Problem Statement

Create an examination timetable where conflicting subjects are not scheduled simultaneously.

Problem Analysis

Variables: Subjects , Domains: Examination slots

Goal: Valid timetable , Constraint: Conflicting subjects $\neq$ same slot

AI Technique : CSP + MRV + Forward Checking

Justification

- CSP represents variables, domains and constraints.
- MRV chooses the most constrained subject, while Forward Checking removes invalid slots.

State Space

Subjects -> MRV Selection-> Assign Slot -> Check Constraint-> Forward Checking-> Next Subject

Variables & Domains

Variables = {AI, DBMS, OS, CN, DAA}

Domains = {Mon-AM, Mon-PM, Tue-AM, Tue-PM}

Heuristic

MRV: Choose the variable with the fewest remaining values.

Pseudocode

1. Select subject using MRV.
2. Select valid slot.
3. Check constraints.
4. Assign slot.
5. Forward Check.
6. If conflict $\rightarrow$ Backtrack.

Example:

Subject	Slot
AI	Mon-AM
DBMS	Mon-PM
OS	Tue-AM
CN	Tue-PM
DAA	Wed-AM

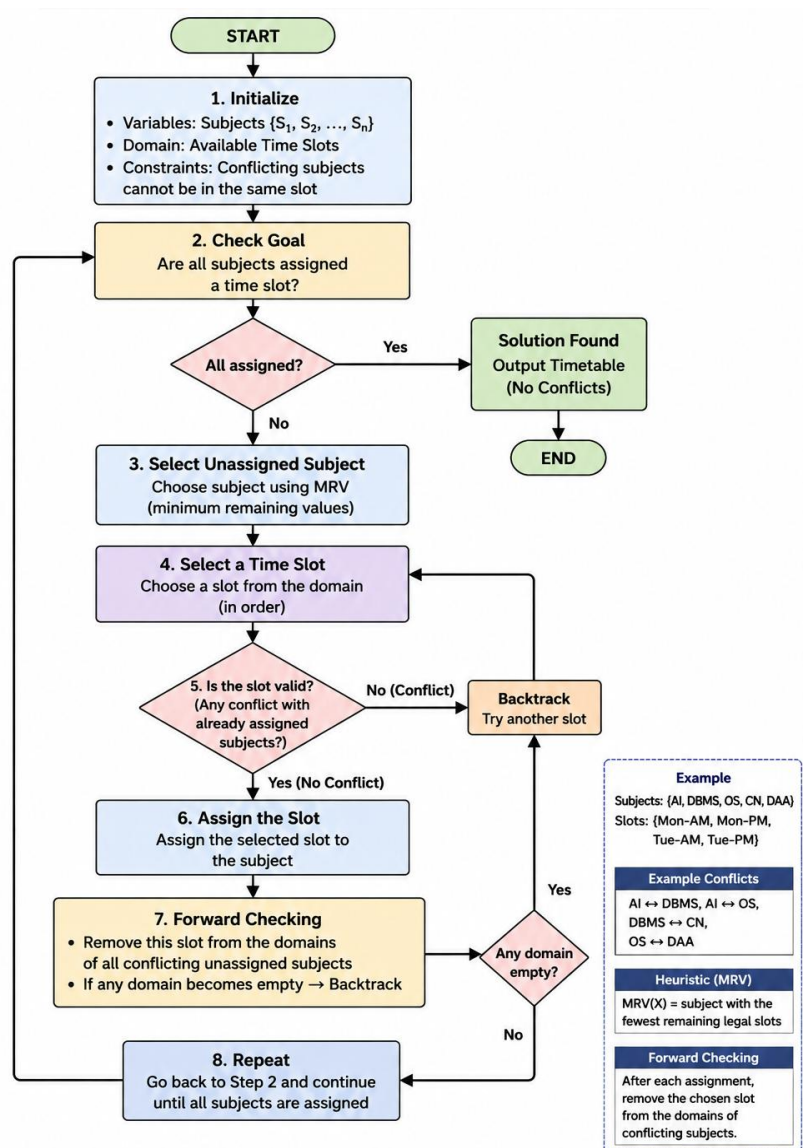
Complexity

Time: $O(d^n)$

Space: $O(n)$

Conclusion

CSP with MRV and Forward Checking efficiently creates a conflict-free timetable.



Q4. Minimax and Alpha-Beta Pruning

Problem Statement

Find the best move for MAX using Minimax
and reduce node evaluation using Alpha-Beta Pruning.

Problem Analysis

Initial: Root game state , Players: MAX and MIN
Terminal states: Leaf nodes , MAX maximizes; MIN minimizes

AI Technique: Minimax + Alpha-Beta Pruning

Justification

- Minimax evaluates possible game outcomes.
- Alpha-Beta eliminates branches that cannot affect the final decision.

Pseudocode

- If terminal → return value.
- MAX → maximum child.
- MIN → minimum child.
- Update α and β .
- If $\alpha \geq \beta$ → prune.

Execution

Left: MIN(5,7)=5
Right: MIN(9,11)=9
Root: MAX(5,9)=9
Best Move = Right

Alpha-Beta

Left value = 5
 $\alpha = 5$

Right MIN:

$9 \rightarrow \beta = 9$
 $10 \rightarrow \alpha = 10$
 $10 \geq 9$

Method	Nodes
Minimax	8
Alpha-Beta	7
Pruned	1

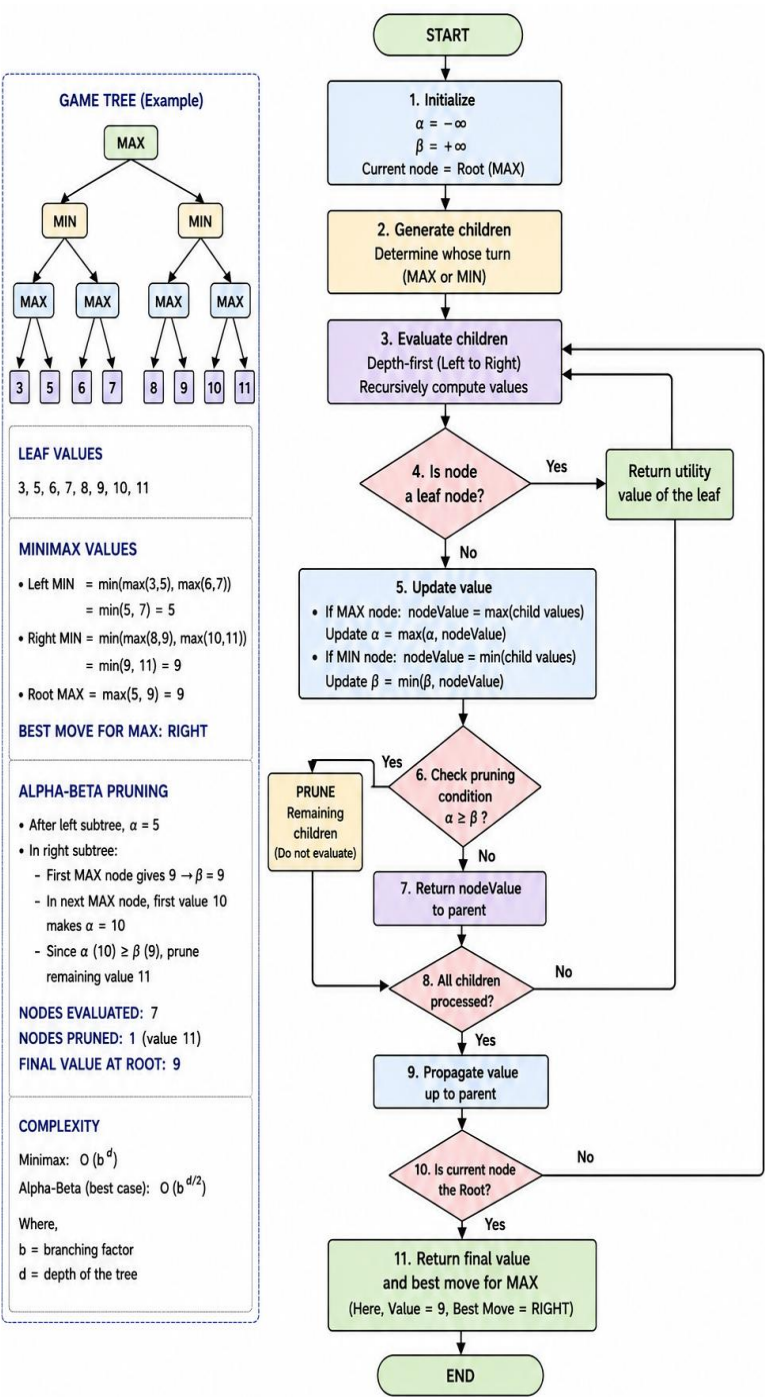
Therefore 11 is pruned

Complexity

Minimax: $O(b^d)$
Alpha-Beta best case: $O(b^{d/2})$

Conclusion

MAX selects the right branch with value 9, while Alpha-Beta reduces unnecessary evaluation.



Q5. Dynamic Warehouse Robot

Problem Statement

Design a search strategy for a warehouse robot that adapts when unexpected obstacles appear.

Problem Analysis

Initial: Robot position , Goal: Destination

Actions: Up, Down, Left, Right , Constraint: Cannot enter obstacles

AI Technique

Online Search Agent : Observe→Act→Update→Replan

Justification

- Online search is suitable for dynamic environments
- Because the robot can discover obstacles and change its route.

State Space

START → A → B → GOAL

START → C → D → GOAL

Variables & Domains

Variables: Robot position, obstacles

Domain: Up, Down, Left, Right

Heuristic

Manhattan distance: $h(n) = |x_n - x_{goal}| + |y_n - y_{goal}|$

Pseudocode

1. Observe environment.
2. Select action.
3. Move robot.
4. Check for obstacles.
5. If obstacle → update map and replan.

Execution

Initial: START → A → B → GOAL

Obstacle at B: START → C → D → GOAL

Complexity

Time: $O(V+E)$

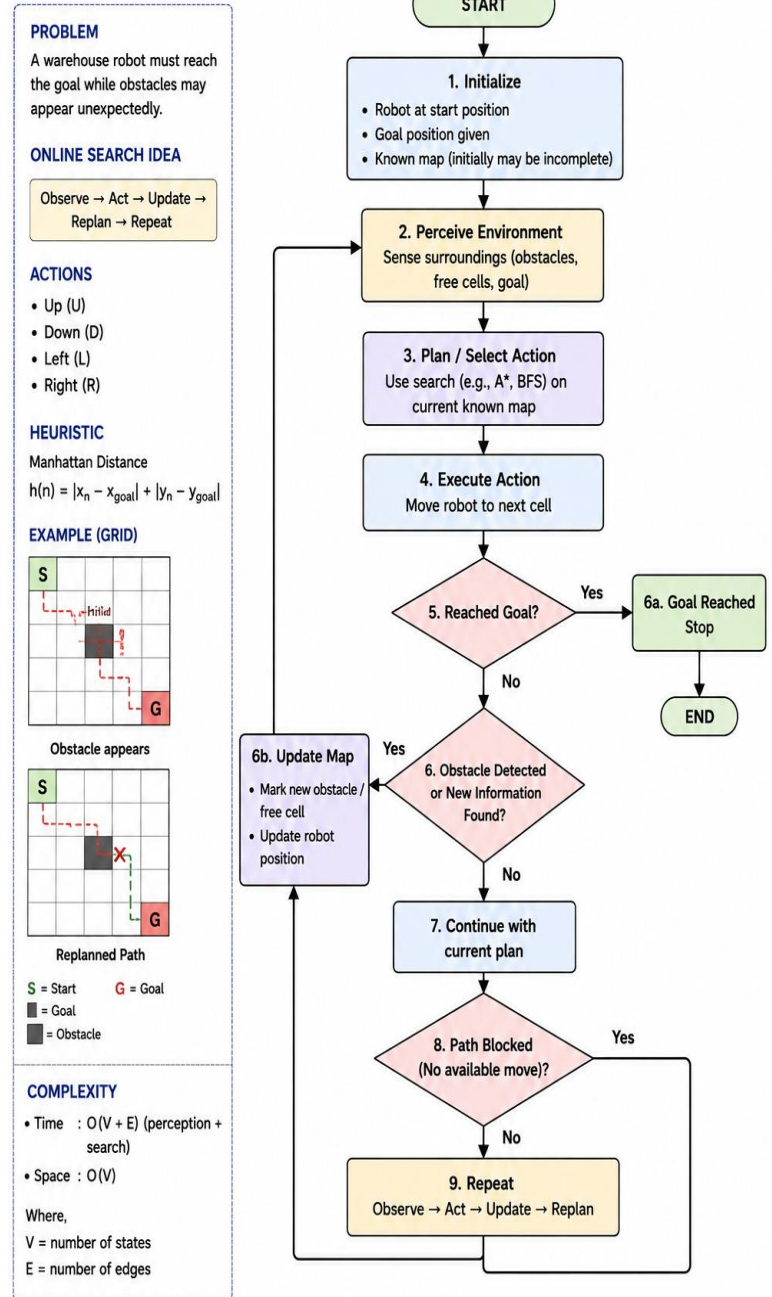
Space: $O(V)$

Advantages

- Handles dynamic environments.
- Supports replanning.
- Adapts to new obstacles.

Conclusion

Online Search allows the warehouse robot to dynamically modify its route and reach the destination safely.



Structure of the Answer – Algorithm Design Challenge

S.No	Sections	Expected Content
1	Problem Statement	Briefly describe the given problem in your own words.
2	Problem Analysis	Identify the initial state, goal state, assumptions, and constraints.
3	AI Technique Selection	Specify the most appropriate AI algorithm/search technique (e.g., Greedy Best-First Search, A*, CSP, Minimax, Local Search).
4	Justification of Algorithm	Explain why the selected algorithm is suitable compared to other search techniques.
5	State Space Representation	Represent the problem using a state-space diagram, search graph, game tree, or constraint graph, as applicable.
6	Variables, Domains, and Constraints (Applicable for CSP problems)	Identify variables, possible domains, and constraints. If not applicable, mention "Not Applicable".
7	Heuristic Function (Applicable for Informed Search)	Define the heuristic function $h(n)$ and explain whether it is admissible or appropriate for the problem. If not applicable, mention "Not Applicable".
8	Algorithm Design / Pseudocode	Write the step-by-step algorithm or pseudocode for solving the problem.
9	Flowchart / Algorithm Diagram	Draw a flowchart, search tree, game tree, or algorithm workflow illustrating the solution.
10	Working / Execution Trace	Demonstrate the execution of the algorithm with at least one example, showing intermediate steps such as node expansion, constraint propagation, or game-tree evaluation.
11	Complexity Analysis	Analyze the Time Complexity and Space Complexity of the proposed algorithm.
12	Advantages and Limitations	Discuss the strengths and limitations of the selected algorithm for the given problem.
13	Conclusion	Summarize the effectiveness of the designed algorithm and suggest possible improvements or alternative approaches.

Criteria	Weightage (%)	Excellent (90–100%)	Good (75–89%)	Average (60–74%)	Poor (<60%)
Problem Analysis and Understanding		13–15% Clearly identifies the problem, initial state, goal state, constraints, and expected outcome.	10–12% – Identifies most problem components with minor omissions.	7–9% – Demonstrates partial understanding with some missing elements.	0–6% – Incorrect or incomplete understanding of the problem.
AI Algorithm Selection & Justification		13–15% – Selects the most appropriate AI algorithm/search technique with strong technical justification.	10–12% – Selects an appropriate algorithm with adequate justification.	7–9% – Selects a partially suitable algorithm with limited justification.	0–6% – Incorrect algorithm selection or no justification.
Algorithm Design / Pseudocode		22–25% – Designs a complete, logical, efficient, and error-free algorithm/pseudocode.	17–21% – Algorithm is mostly correct with minor logical errors.	12–16% – Algorithm is partially complete with several logical errors.	0–11% – Algorithm is incomplete, incorrect, or difficult to understand.
Application of AI Concepts (Greedy, A*, CSP, Minimax, Alpha-Beta, etc.)		13–15% – Correctly applies the relevant AI concepts and techniques	10–12% – Applies AI concepts correctly with minor mistakes.	7–9% – Demonstrates limited application of AI concepts.	0–6% – Unable to apply the appropriate AI concepts or techniques.

		throughout the solution.			
--	--	--------------------------	--	--	--

Diagram / State Space / Search Tree Representation		9–10% – Accurate, complete, and well-labeled diagram supporting the solution.	7–8% – Diagram is mostly correct with minor omissions.	5–6% – Diagram is partially complete or lacks clarity.	0–4% – Diagram is missing or incorrect.
Complexity Analysis and Solution Efficiency		9–10% – Correctly analyzes time and space complexity and proposes an efficient solution.	7–8% – Complexity analysis is mostly correct with minor errors.	5–6% – Partial complexity analysis or limited discussion of efficiency.	0–4% – No meaningful complexity analysis or highly inefficient solution.
Originality, Critical Thinking, and Presentation		9–10% – Demonstrates excellent originality, logical reasoning, organized presentation, and explores optimized/alternative solutions.	7–8% – Demonstrates good critical thinking with a clear presentation.	5–6% – Shows limited originality and acceptable presentation.	0–4% – Poor presentation with little or no critical thinking.